Technical University of Cluj-Napoca
Faculty of Electronics, Telecommunications and Information Technology
2024

Microcontrollers

Functional Thermostat

Supervisor:

Mirela Olteanu

Student:

Horgoș Raul-George

Table of Contents

Project Overview **3**

1.1 The block scheme 3
1.2. Types of temperature sensors 4

1.2.1. The infrared temperature sensor 4
1.2.2. Resistive temperature sensor 4
1.2.3. Semiconductor temperature sensor 5
1.2.4. Thermocouple 5

1.3. The implementation with the ADC 8

2. Calculations **11**

3. Display using an LCD **13**

3.1. Table of comparison of displays 15
3.2. The implementation 16
3.3. The microcontroller 17
3.4. The keyboard and relay 20

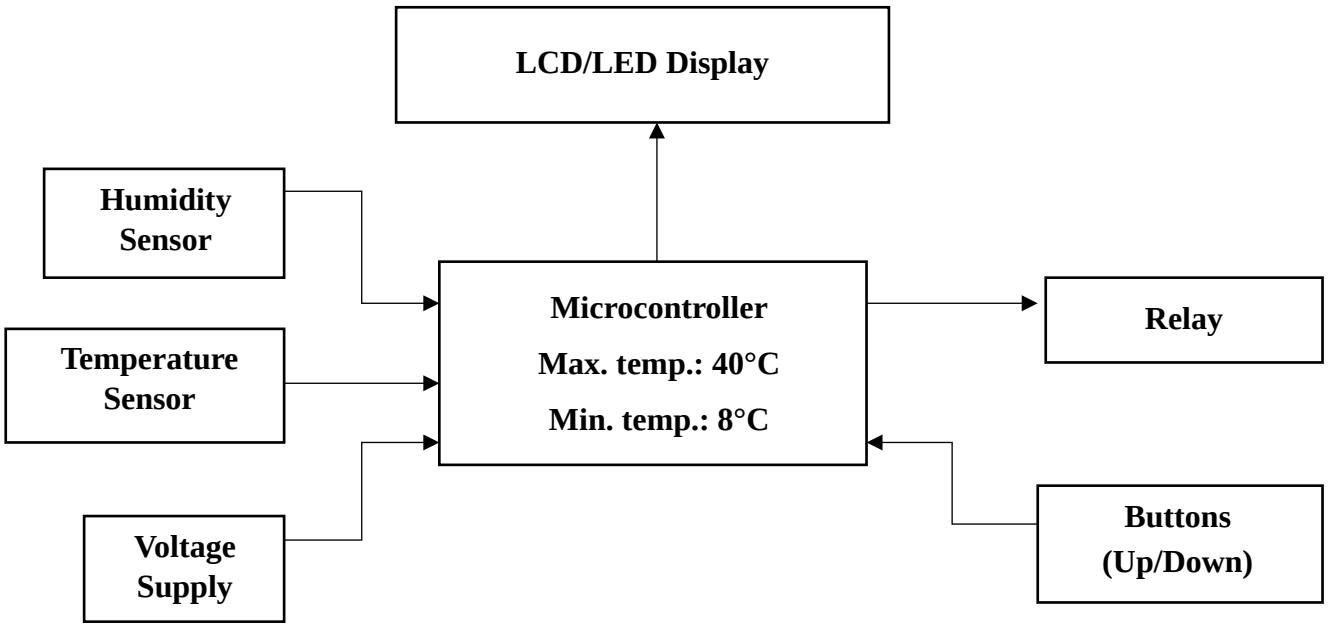
4. Bibliography **24**

5. Appendix **25**

1. Project overview

Traditional thermostats have long relied on mechanical components with limited functionality and manual operation. However, advancements in microcontroller technology have paved the way for smarter, more efficient solutions. By harnessing the power of microcontrollers, we have created a thermostat system that is adaptive, programmable, and environmentally conscious.

1.1 The block scheme



The temperature sensor is connected to the microcontroller, which reads the temperature and controls the thermostat settings and send information to the microcontroller from where it will display it on the LCD/LED display.

The microcontroller is also connected to a LCD/LED Display, a relay (it can be a mechanical one or a Solid-state relay) and buttons for the user input. The Display is used to show the user the information form the sensor that is processed in the microcontroller and give information about the current temperature. The relay is used to control the heater or cooler.

The buttons are the only part of the circuit that a user has access to, and it is used to control the wanted temperature by each individual.

The volage supply is used to power the whole circuit.

1.2. Types of temperature sensors

1.2.1. The infrared temperature sensor:

Non-contact temperature sensors, commonly referred to as infrared temperature sensors, operate by sensing the heat radiation that an object emits. The amount of infrared radiation emitted by any object above absolute zero (-273.15°C) is directly proportional to that object's temperature. This radiation is detected by infrared temperature sensors, which then transform it into an electrical signal that is processed to find the object's temperature. Normally measuring from -50°C to 2000°C, infrared temperature sensors are widely used in a variety of industries and applications where non-contact temperature measurement is necessary.



Figure 1 Infrared Sensor

1.2.2. Resistive temperature sensor:

A resistive temperature sensor (RTD) is a type of temperature sensor that determines changes in a metal wire's or element's resistance. Since most metals' resistance rises with temperature, an RTD takes the metal element's resistance and converts it to a temperature reading. The most popular kind of RTD is the platinum RTD, which is made up of a thin platinum wire that is either deposited on a ceramic substrate or wound into a coil. Accurate temperature measurement is made possible by the platinum wire's predictable change in resistance with temperature. The majority of metals used in RTDs have a temperature-dependent increase in resistance, though this relationship is not always linear.

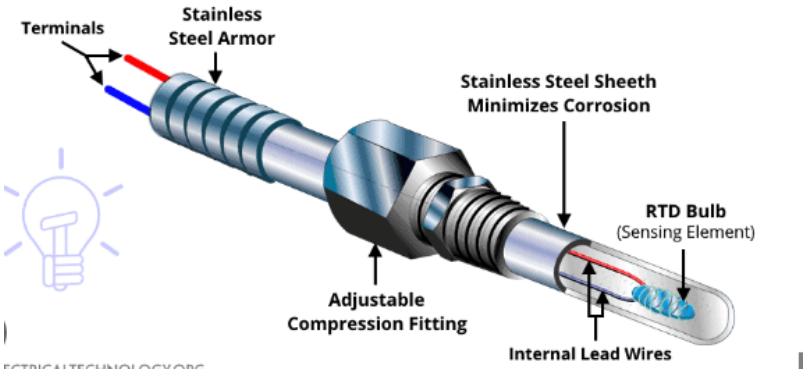


Figure 2 RTD

1.2.3 Semiconductor temperature sensor:

Semiconductor temperature sensors measure temperature by using a diode or transistor whose voltage changes with temperature. The silicon bandgap temperature sensor and the forward voltage drop temperature sensor are the two most popular varieties of semiconductor temperature sensors.

The silicon bandgap temperature sensor operates by taking advantage of the silicon bandgap energy's temperature dependence. The bandgap energy decreases with temperature, resulting in a reduction in the voltage drop across the silicon diode.

The voltage drop across a forward-biased junction of a transistor or diode is measured by the forward voltage drop temperature sensor, commonly referred to as a junction temperature sensor.

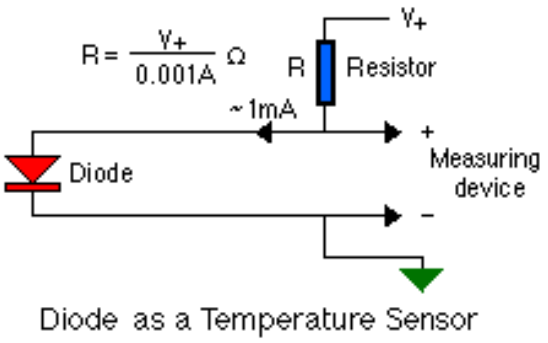


Figure 3 Semiconductor Sensor

1.2.4. Thermocouple:

By joining two different metal wires at one end and connecting them to a thermocouple thermometer or another suitable device at the other, a thermocouple can be used as a temperature sensor. Thermocouples can measure temperatures over a wide range when they are configured properly.

Conversely, the Seebeck effect is a phenomenon in which a temperature differential between two dissimilar electrical conductors or semiconductors results in the generation of a voltage difference between them.

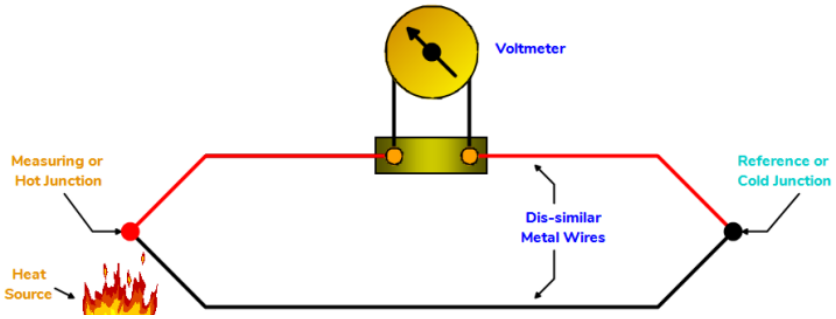


Figure 4 Thermocouple

Table 1 Comparison of the types of sensors

	Infrared temperature sensor	**Resistive temperature sensor**	**Semiconductor temperature sensor**	**Thermocouple**
Temperature range:	-50°C to 3000°C	-50°C to 300°C	-55°C to 150°C	-200°C to 2300°C
Price/availability:	More expensive ($50 to several thousand dollars)	Generally more expensive than thermocouples	Moderately priced (a few dollars to around $50)	Generally the most affordable
Accuracy:	±1°C to ±5°C	±0.1°C to ±1°C	±0.1°C to ±1°C	±0.5°C to ±2°C
Response time:	Fast	Moderate to slow	Moderate to fast	Very fast
Sensitivity	High	High	High	Low

For this project I chose the semiconductor-based sensor.

Some of the reasons for choosing this category are:

- **Accuracy:** Semiconductor sensors can provide highly accurate temperature measurements, ensuring precise control of the room temperature.
- **Fast Response Time:** They have a rapid response time, allowing the thermostat to quickly adjust heating or cooling systems in response to temperature changes, leading to improved comfort and energy efficiency.
- **Compact Size:** Semiconductor sensors are typically small and compact, making them suitable for integration into the design of the thermostat without taking up much space.
- **Low Power Consumption:** Many semiconductor sensors have low power consumption, which is beneficial for battery-powered or energy-efficient thermostat designs, helping to prolong battery life or reduce electricity usage
- **Cost-Effectiveness:** Semiconductor sensors are often cost-effective compared to other types of sensors, making them a practical choice for mass-produced thermostat projects.

Table 2 Comparison between sensors

Semiconductor temperature sensor	Temperature range	Accuracy in measurement	Price [RON]
LM35	-55°C - 150°C	±0.4°C	14.5
LM75	-55°C - 125°C	±2°C	28.5

MCP9700	40°C to 125°C	±2°C	2
AD590M	-55°C - 150°C	±0.5°C	28.5
DS18B20	-55°C - 125°C	±0.5°C	45

For the sensor I chose the LM35 for this project.

Reasons for choosing LM35 temperature sensor:

1. **Accuracy:** The LM35 is renowned for its exceptional precision in temperature readings, usually within ±0.5°C at ambient temperature. This precision guarantees accurate thermostat control, which raises comfort and lowers energy use.
2. **Wide Temperature Range:** Typically, the LM35 is capable of measuring temperatures between -55°C and +150°C. Its wide range makes it appropriate for a number of uses, including as controlling room temperature.
3. **Ease of Use:** The LM35's simple design and three-pin packaging make it simple to use and incorporate into electronic circuits. Its reduced requirement for external components simplifies the thermostat project's overall design.
4. **Linear Output:** The LM35 makes it easier to connect with microcontrollers or other electrical circuits by providing a linear output voltage that is proportionate to the temperature being measured. It is easier to develop simple temperature calibration and control algorithms thanks to this linear relationship.
5. **Availability & Cost-Effectiveness:** The LM35 temperature sensor offers both availability and cost-effectiveness. Widely available from various electronics suppliers, it's easy to procure for project development and production. Additionally, its affordability compared to other sensors with similar accuracy and features makes it suitable for both prototype development and mass production of thermostat projects.

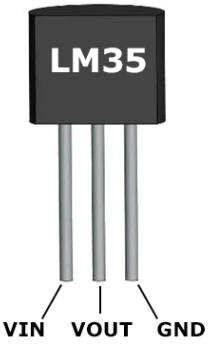


Figure 5 LM35 sensor

1.3. The implementation of the temperature sensor with an ADC and simulating the circuit in Proteus

The components of the schematic are:

- The temperature sensor (LM35)
- The operational amplifier (AD8032)
- An Analog to Digital converter (ADC0808)
- The microcontroller 8051

The temperature sensor:

The output will be a voltage that is linearly proportional to the temperature indicated in degrees Celsius because the sensor has a direct calibration in degrees Celsius. As a result, the output voltage will change by 10 mV for every degree Celsius that the temperature varies.

This sensor produces a voltage corresponding to a temperature, which is then converted into digital form and transmitted to the 8051 microcontroller

The operational amplifier:

The LM7341 is a precision operational amplifier (op-amp) manufactured by Texas Instruments. It is designed for applications requiring high accuracy, low offset voltage, and low quiescent current consumption. Some of the main characteristics of this operational amplifier are:

- Low Offset Voltage: The LM7341 offers low input offset voltage, typically around 150 µV, which minimizes errors in precision applications.
- Rail-to-Rail Output: The op-amp provides rail-to-rail output swing, allowing it to operate with input and output signals that approach the power supply rails.
- Unity Gain Stability: It is designed for unity gain stability, ensuring stability and performance in unity gain configurations without the need for external compensation components.
- Temperature Stability: The LM7341 offers good temperature stability over the specified operating temperature range, ensuring consistent performance across temperature variations.

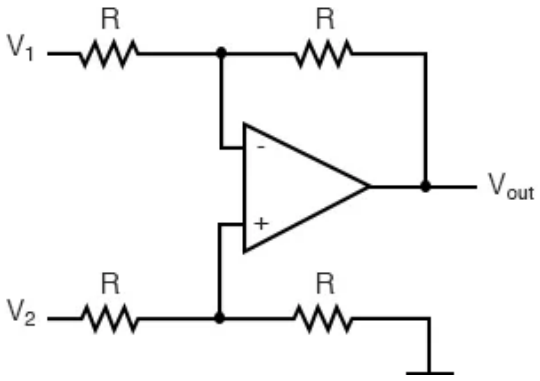


Figure 6 Generic schematic of an differential amplifier

The analog to digital converter:

An 8-bit analog-to-digital converter (ADC) with a wide range of applications is the ADC0808. Its purpose is to transform analog signals into digital signals so that a microcontroller or other digital device can process them.

Applications of the ADC0808 include data acquisition systems, instrumentation, process control, industrial automation, and sensor interfacing, among others. Its simplicity, ease of use, and reliability have made it a popular choice for many analog-to-digital conversion tasks over the years.

The ADC0808 has an input voltage range of 0V to 5V and can convert analog signals at a rate of up to 100,000 samples per second. It has a resolution of 8 bits, which means it can represent analog signals in 256 discrete levels.

In a thermostat, the ADC0808 may be used to convert the analog voltage signal from a temperature sensor into a digital signal that can be processed by the microcontroller.

ADC0808 temperature measurement and control applications are popular because of its low cost, high-speed performance, and user-friendliness. Its 8-bit resolution is usually enough for the majority of temperature control applications, and a variety of sensors can be used with it due to its 0V to 5V input voltage range.

Some of the main characteristics of this microcontroller are:

- It is easy to interface with all microcontrollers.
- It has a resolution of 8 bits.
- It operates with a 5V unipolar power supply.
- It has a low power consumption around 15mW
- It has an 8 bit converter with 8 multiplexed channels.

The device uses the address lines ADD A, ADD B, and ADD C to select the input channel. The analog signal from the LM35 temperature sensor only requires one input in the circuit made specifically for my use. I'll select IN0 as this input, and I get the following by consulting the table of matching values for input I0: Low, low, low, ADD A, B, and C.

The reference voltage is set by using the Vref+ and Vref- pins, marked as 12 and 16 in the diagram. Depending on the value of this reference voltage, the absolute resolution is established, which is the same as the voltage of the least significant bit (LSB). For a Vref+ of 5V and a Vref- of 0V (VFS = 5-0 = 5V), the absolute resolution is calculated as follows: Rabs = VLSB = VFS/2n = 5/28 = 20 mV.

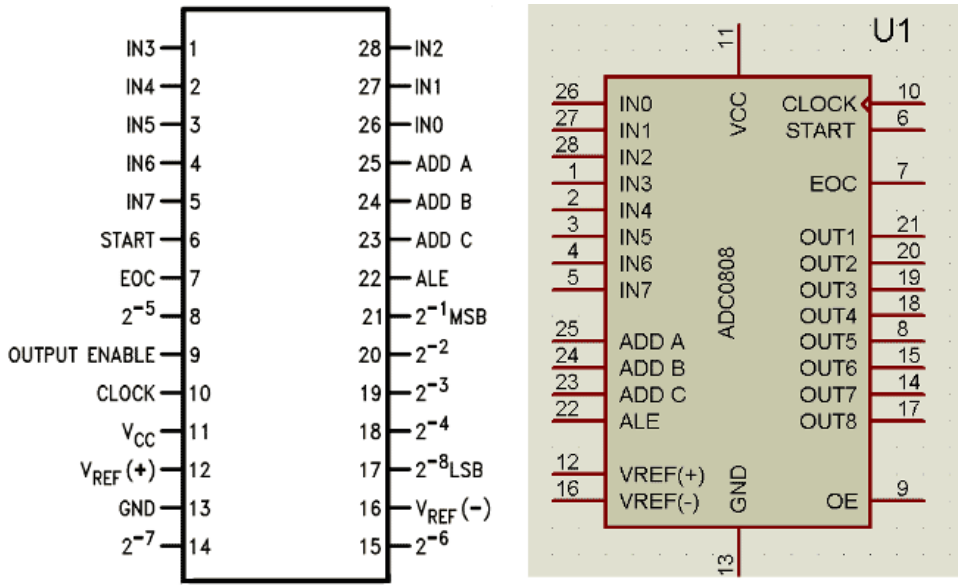


Figure 7 ADC

The ADC0808 typically comes in a 20-pin dual in-line package (DIP). Here's a brief overview of the pins and their functions:

1. **VCC (Pin 20):** This pin is connected to the positive supply voltage (VCC), typically between +5V to +15V.
2. **VREF (Pin 19):** VREF is the reference voltage input pin. It sets the maximum analog input voltage range for conversion. It can be externally connected to a reference voltage or tied to VCC for full-scale input range.
3. **AGND (Pin 18):** AGND is the analog ground pin, connected to the ground reference for the analog circuitry.
4. **Analog Inputs (Pins 1-8):** These pins are the analog input channels. Each channel accepts the analog input voltage to be converted to digital format.
5. **EOC (Pin 9):** EOC stands for End of Conversion. This output pin indicates the end of the conversion cycle. It goes low when the conversion is complete.
6. **CLK IN (Pin 10):** This pin is the clock input for the conversion process. It provides the timing signal for the successive approximation process.
7. **START (Pin 11):** The START pin initiates the conversion process when brought low. It triggers the ADC to start converting the analog input.
8. **A0 - A2 (Pins 12-14):** These pins are address inputs used for chip select in multiple ADC configurations.
9. **CS (Pin 15):** CS stands for Chip Select. It enables the ADC when brought low and disables it when high.
10. **WR (Pin 16):** WR stands for Write. This input pin is used to initiate a write operation when converting the analog input to digital format.
11. **RD (Pin 17):** RD stands for Read. This input pin is used to initiate a read operation of the converted digital data.
12. **DB0 - DB7 (Pins 2-9):** These pins are the parallel digital outputs. They provide the 8-bit digital output representing the converted analog input.

13. **DGND (Pin 20):** DGND is the digital ground pin, connected to the ground reference for the digital circuitry.

2. Calculations

This stage's objective is to increase the temperature sensor's output voltage and adjust its level in accordance with the numbers shown below.

Table 3 Specifications

Temperature	Output voltage
8°C	0V
40°C	5V

For the schematic shown before we get the output voltage formula

$$Vout = \frac{R_f}{R_1} * (V_2 - V_1) \text{ (2.1)}$$

The resolution of the ADC has the formla:

$$V_{LSB} = \frac{V_{ref}}{2^n} = \frac{5V}{2^*} = 19.53mV \text{ (2.2)}$$

The output of the LM35 sensor is given by the result of:

$$V_{OUT} = \frac{10mV}{°C} * T \text{ (2.3)}$$

At 80°C, the output voltage is 0.4V, while at 40°C, it is 80mV. Therefore, when the temperature is equal to 8°C, the reference voltage must be 80mV in order to set the amplifier's output at 0V. A resistive divider is utilized to obtain this voltage. Consequently, we have the equation:

$$V_{ref} = \frac{R_a}{R_a + R_b} * 5V = 80mV \text{ (2.4)}$$

$$\frac{R_a}{R_b} = 61.5 \ (where\ R_a\ is\ 100\Omega\ and\ R_b\ is\ 6.15k\Omega) \text{(2.5)}$$

The gain has the following formul:

$$A_v = \frac{R_f}{R_1} = \frac{V_{OA}}{V_2 - V_1} \text{ (2.6)}$$

$$\frac{R_f}{R_1} = \frac{5}{0.4 - 0.08} = 15.625\ k\Omega\ (in\ the\ case\ of\ 40°C) \text{ (2.7)}$$

For the amplification stage shown in the figure below in the Figure we should get the results:

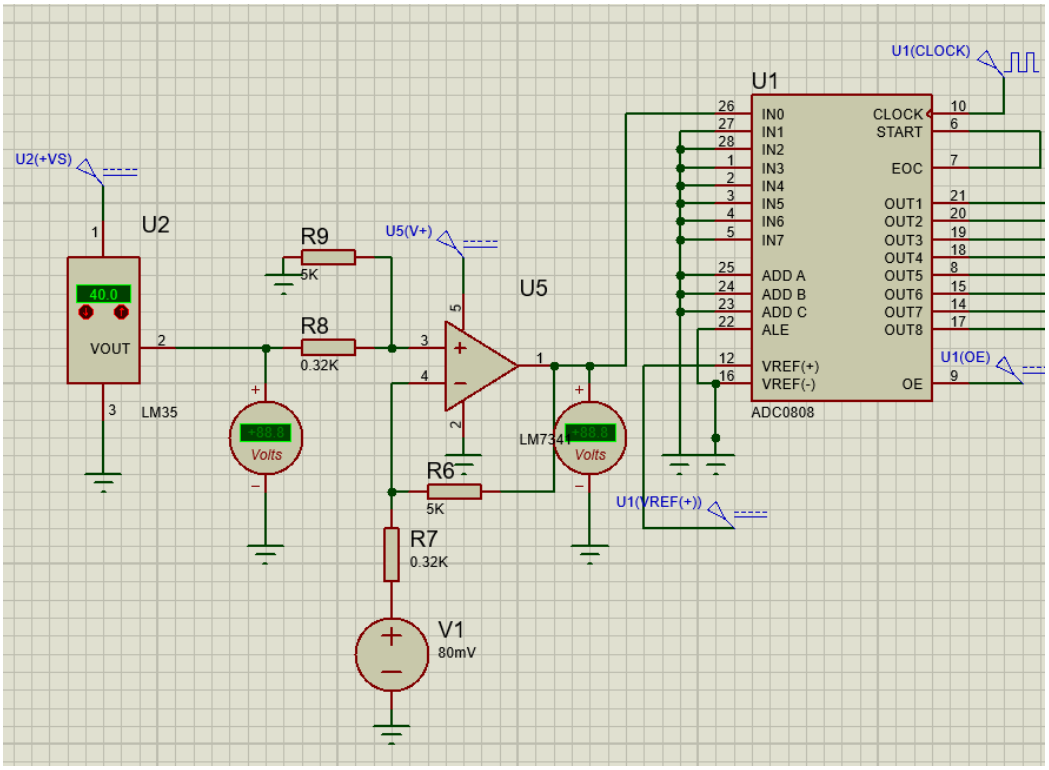


Figure 8 The circuit only with the ADC

Table 4 The results obtained using only the ADC

Temperature	Amplifier output	ADC output
8°C	0.04V	02h
40°C	4.95V	FCh

3. Display using an LCD

A liquid-crystal display (LCD) is a flat-panel display or other electronically modulated optical device that uses the light-modulating properties of liquid crystals combined with polarizers. Liquid crystals do not emit light directly but instead use a backlight or reflector to produce images in color or monochrome.

Optical imaging or to call it the working principle of the liquid crystals is that, a liquid crystal cell is placed between two layers of photoconductor. Light is applied to the photoconductor, which increases the material's conductivity. This causes an electric field to develop in the liquid crystal corresponding to the intensity of the light.

Displays for a small number of individual digits or fixed symbols can be implemented with independent electrodes for each segment. In contrast, full alphanumeric or variable graphics displays are usually implemented with pixels arranged as a matrix consisting of electrically connected rows on one side of the LC layer and columns on the other side, which makes it possible to address each pixel at the intersections. The general method of matrix addressing consists of sequentially addressing one side of the matrix, for example by selecting the rows one-by-one and applying the picture information on the other side at the columns row-by-row.

LCDs are manufactured in cleanrooms borrowing techniques from semiconductor manufacturing and using large sheets of glass whose size has increased over time. Several displays are manufactured at the same time, and then cut from the sheet of glass, also known as the mother glass or LCD glass substrate.

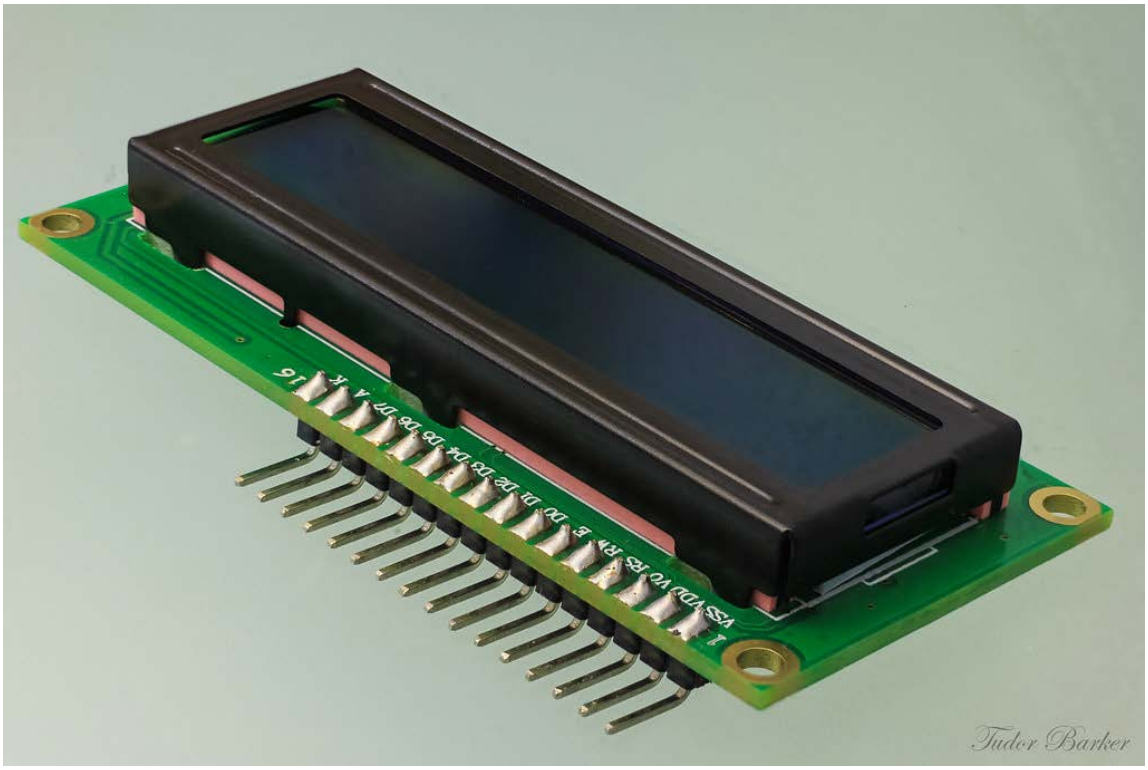


Figure 9 The LCD display

The most commonly used Character based LCDs are based on Hitachi’s HD44780 controller or other which are compatible with HD44580. This LCD module can display up to 16 characters per row and it has 2 rows of characters.

The LM016L module can be interfaced with a wide range of digital devices including microcontrollers having a basic parallel interface, including of a data bus, control signals, and power supply connections.

The module also includes an LED backlight for improved visibility in low-light conditions.

The R/W signal indicates whether the data is being written to or read from the display, whereas the RS signal indicates if the data is character or command data. Data is latching onto the signal's falling edge through the E signal.

The pins of the LM016L are:

Sure, here's a concise breakdown:

1. VSS: Ground.
2. VDD: Power supply (typically +5V).
3. VO: Contrast adjustment.
4. RS: Register Select (0 for commands, 1 for data).
5. RW: Read/Write (0 for write, 1 for read).
6. E: Enable (clocks data).
7. D0-D7: Data lines.
8. A: Anode of the LED backlight.
9. K: Cathode of the LED backlight.

The ASCII – Binary Character Table



Figure 10 The ASCII to Binary table

3.1. Table of comparison of displays

Table 5 Comparison between displays

	LM016L	SSD1306	TFT-LCD
Display Type	Alphanumeric	Graphics	Graphics
Technology	LCD	OLED	TFT (Thin Film Transistor)
Character Size	16x2	Varies	(Not applicable)
Interface	Parallel	I2C, SPI	SPI, Parallel, I2C
Power Consumption	Moderate	Low	Moderate to High
Cost	Low to Moderate	Moderate	Moderate to High

3.2. The implementation:

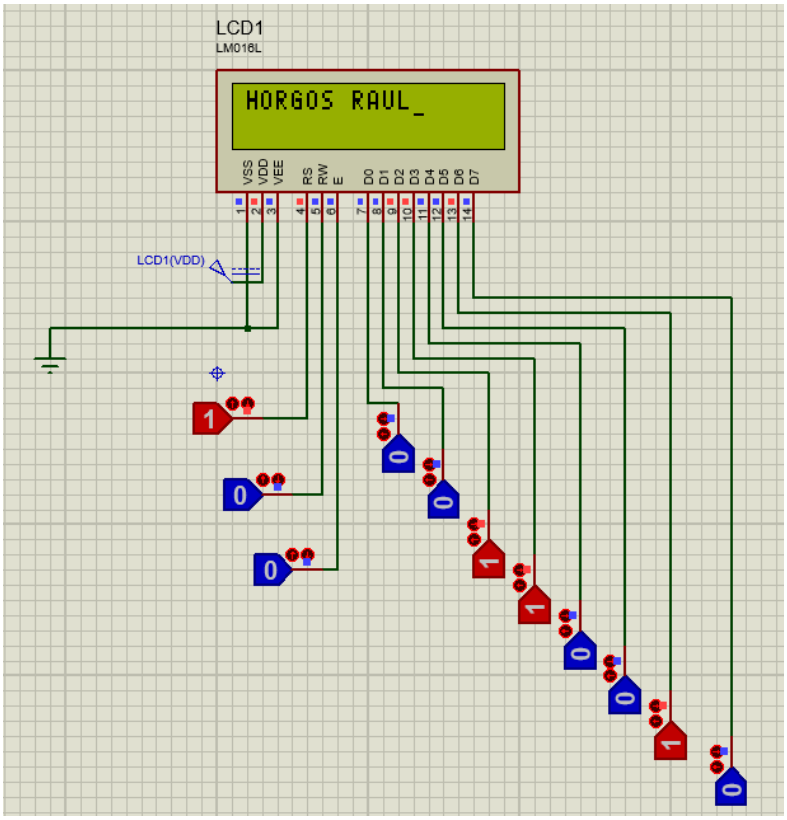


Figure 11 The usage of the display

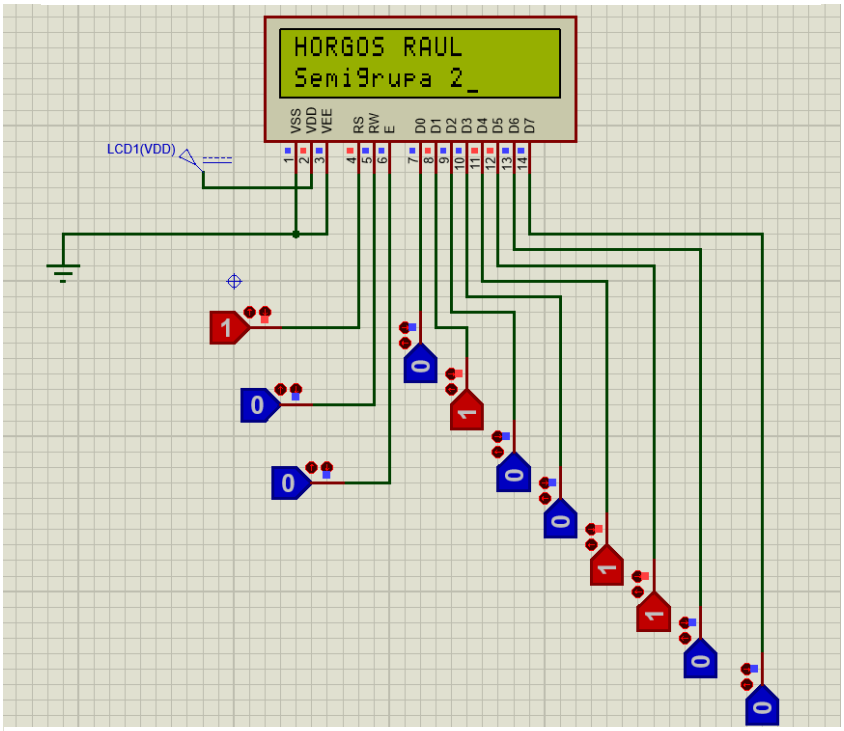


Figure 12 The final form of the display using logic gates

3.3. The microcontroller

For this project I used the 8051 microcontroller that was first introduced by Intel in the early 1980s. It is one of the most widely used microcontrollers in the industry due to its simplicity, versatility and low costs. The 8051 has Harvard-inspired programs and data memory spaces. Some of the main features of the 8051 microcontroller are:

- 8-bit CPU: The 8051 is an 8-bit microcontroller, which means that it can process data in 8-bit data.
- On-chip memory: The 8051 includes on-chip ROM (Read-Only Memory) and RAM (Random Access Memory) to store the program code and data respectively.
- Programmable I/O: The 8051 has 4 ports (Port 0, Port 1, Port 2, and Port 3), each of which can be programmed as input or output.
- Timers and counters: The 8051 has two 16-bit timers/counters that can be used to generate accurate time delays or to count external events.
- Interrupts: The 8051 supports both hardware and software interrupts, which can be used to respond to external events or to perform time-critical operations.
- Serial communication: The 8051 has a built-in UART (Universal Asynchronous Receiver/Transmitter) that supports serial communication at different baud rates.
- Low power consumption: The 8051 has low power consumption and can be used in battery-powered devices.
- Instruction set: The 8051 has a rich instruction set with over 100 instructions, including arithmetic and logic instructions, branch instructions, and instructions for I/O operations.

To compare this microcontroller to others we have the following table:

Table 6 Comparison between microcontroller types

Feature	8051	8052	8031	8032	80552
Internal RAM	128 bytes	256 bytes	None (External)	256 bytes	Variable (External)
Internal ROM	4 KB	8 KB	None (External)	4 KB	Variable (External)
Additional Ports	P2, P3	P2, P3	None	None	Variable
Clock Frequency	Up to 12 MHz	Up to 12 MHz	Up to 12 MHz	Up to 12 MHz	Higher than the 8051

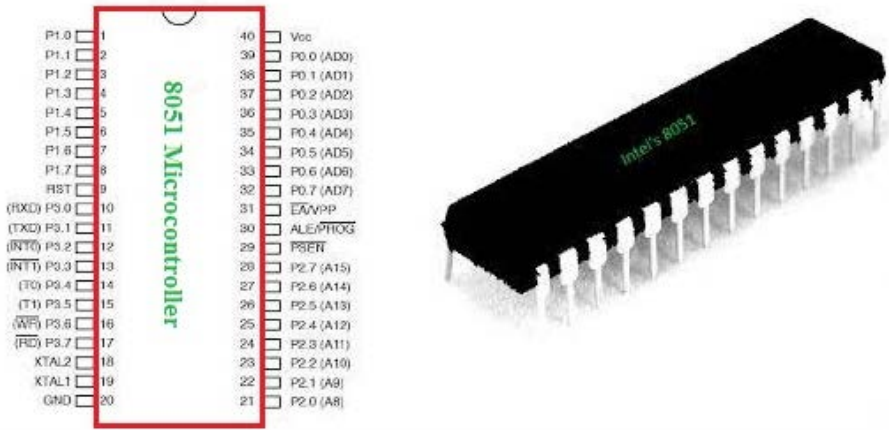


Figure 13 The 8051 microcontroller

The 8xC51 and 8xC52 contain a 128 × 8 RAM and 256 × 8 RAM respectively, 32 I/O lines, three 16-bit counter/timers, a six-source, four-priority level nested interrupt structure, a serial I/O port for either multi-processor communications, I/O expansion or full duplex UART, and on-chip oscillator and clock circuits.

Therefore, we get the schematic of the ADC connected to the microcontroller and then the connection from the 8051 to the LCD display.

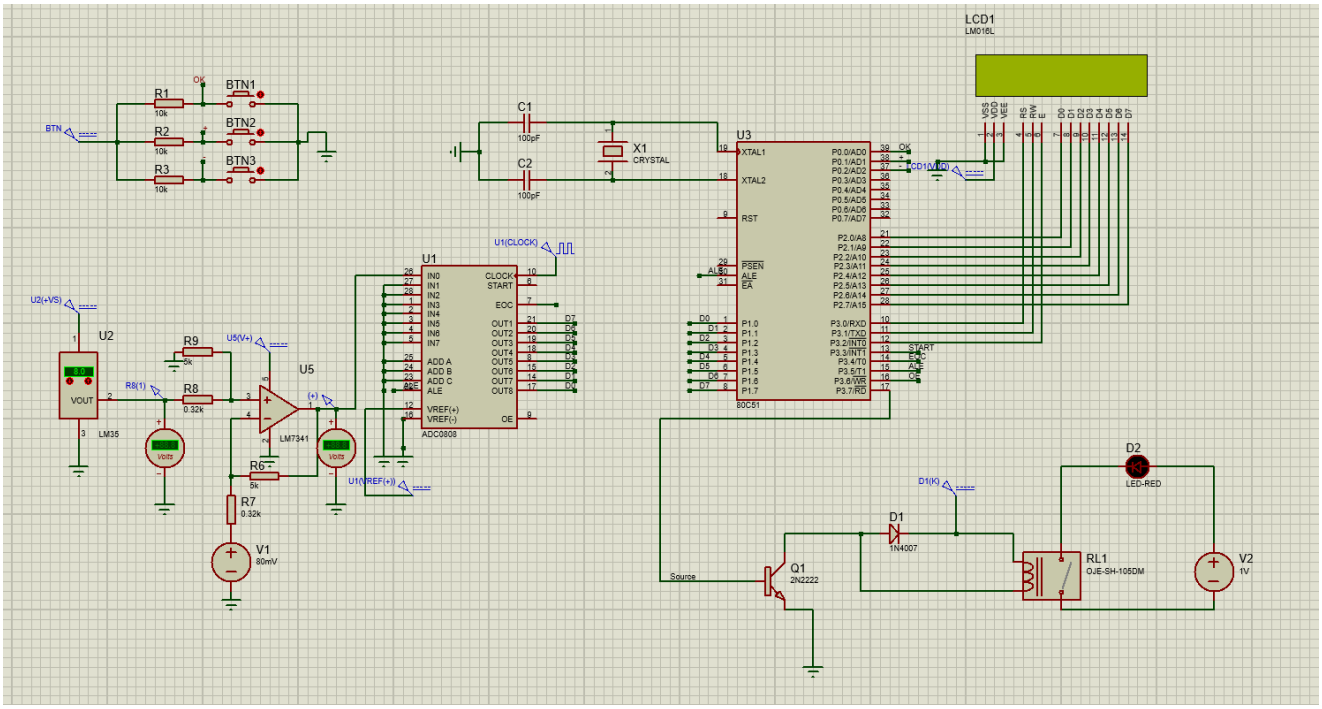


Figure 14 The final schematic

3.4. The keyboard:

The keyboard consisting of three buttons (+, -, OK) and is useful in a thermostat with a microcontroller.

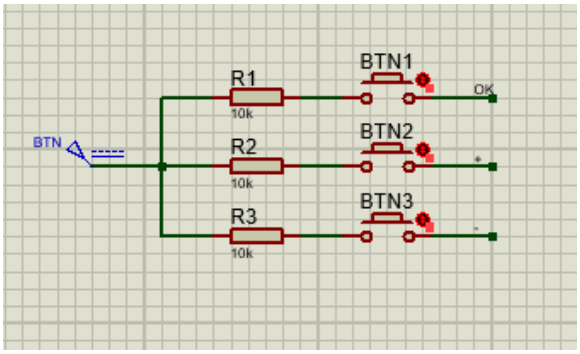


Figure 15 The keyboard

OK Button

- Function: Confirms and sets the desired temperature.
- Operation:
- Press once to enter temperature setting mode.
- Adjust temperature with + and - buttons, then press OK again to save.

+ Button

- Function: Increases the set temperature.
- Operation:
- Press to increase the temperature by one unit.
- Hold to continuously increase the temperature.

- Button

- Function: Decreases the set temperature.
- Operation:
- Press to decrease the temperature by one unit.
- Hold to continuously decrease the temperature.

The relay:

The relay is used to ensure the relay is properly installed in a safe, accessible location, away from moisture and excessive heat, disconnect power before wiring the relay to prevent electric shock or short circuits and verify proper operation by setting the thermostat to trigger the relay and observing the activation of the heating or cooling system.

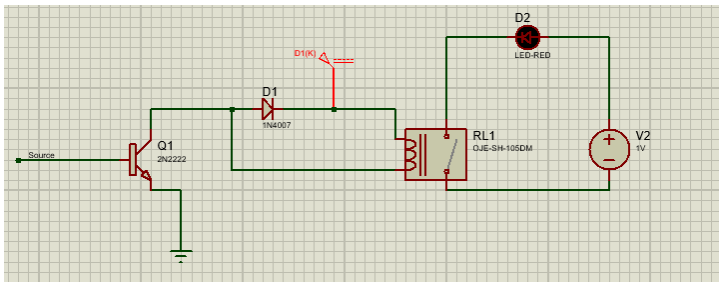
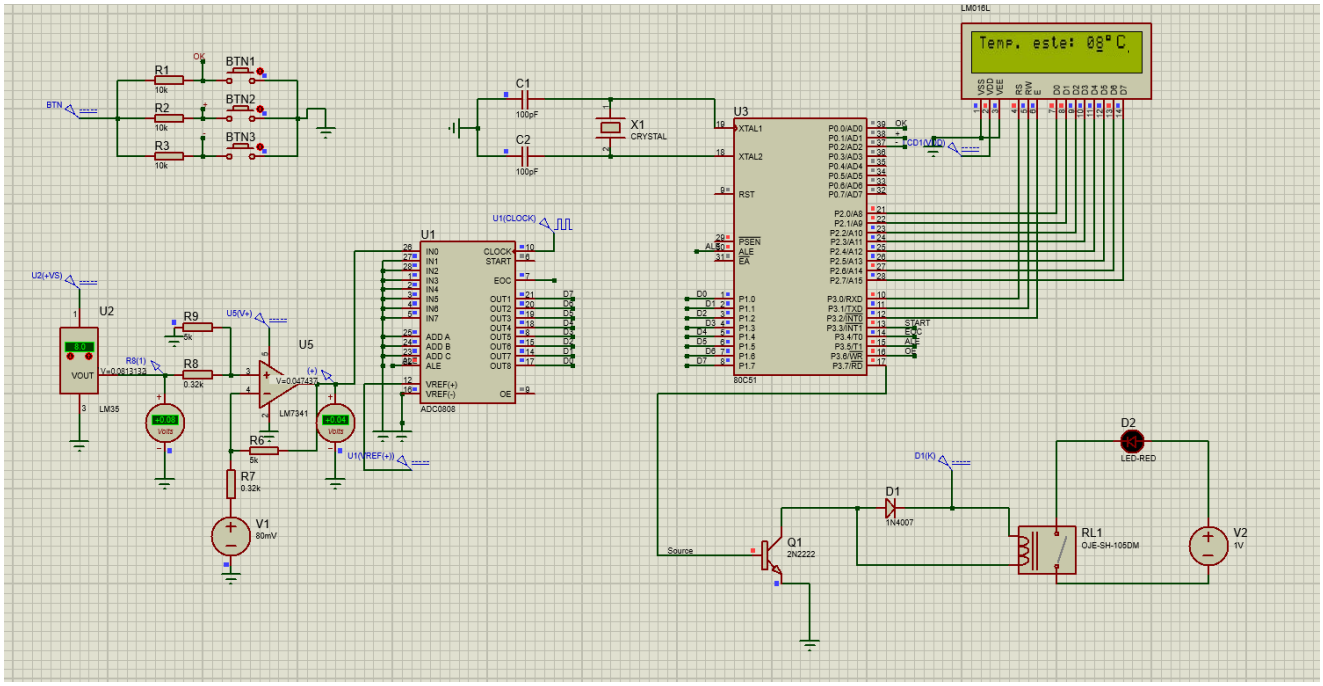
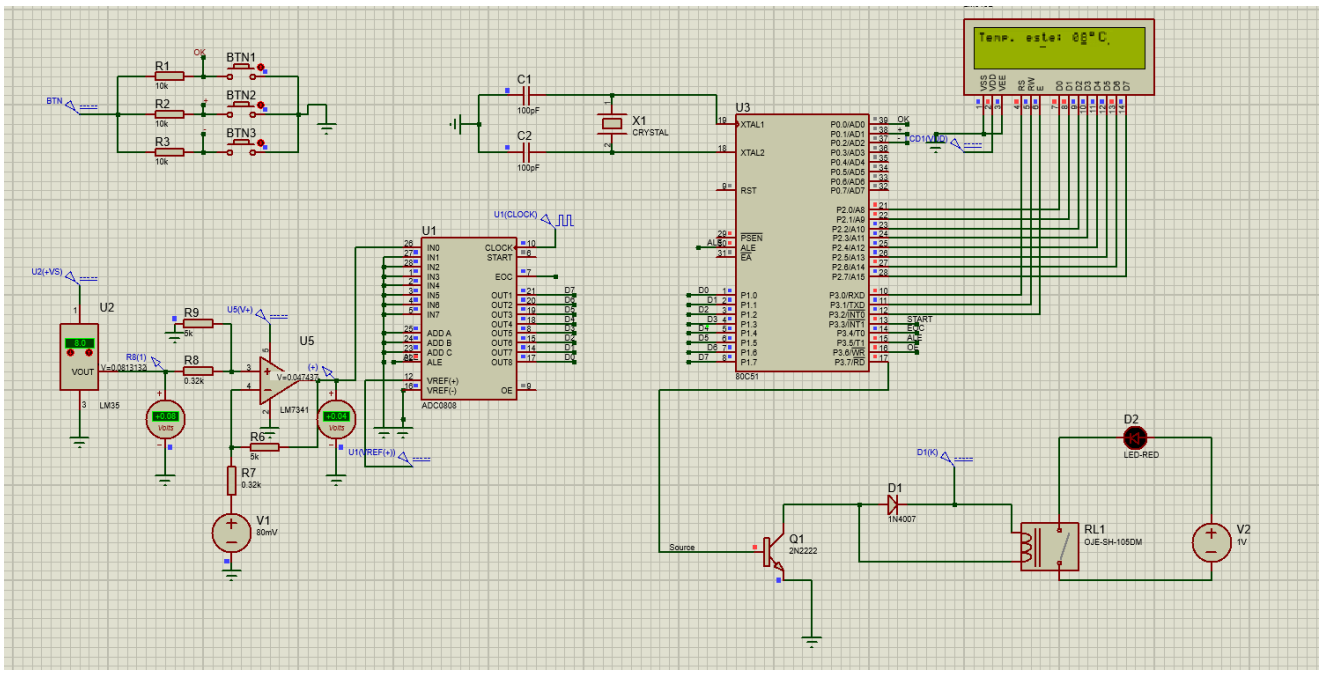


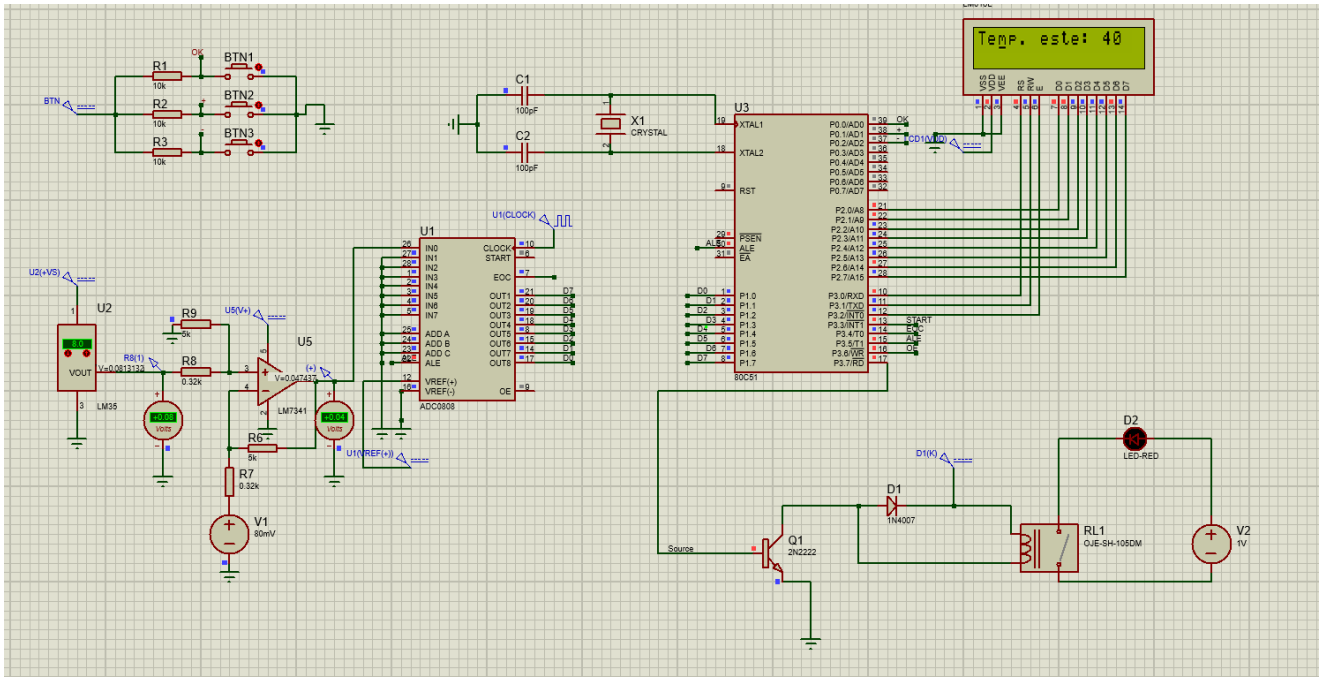
Figure 16 The relay

The final Schematic

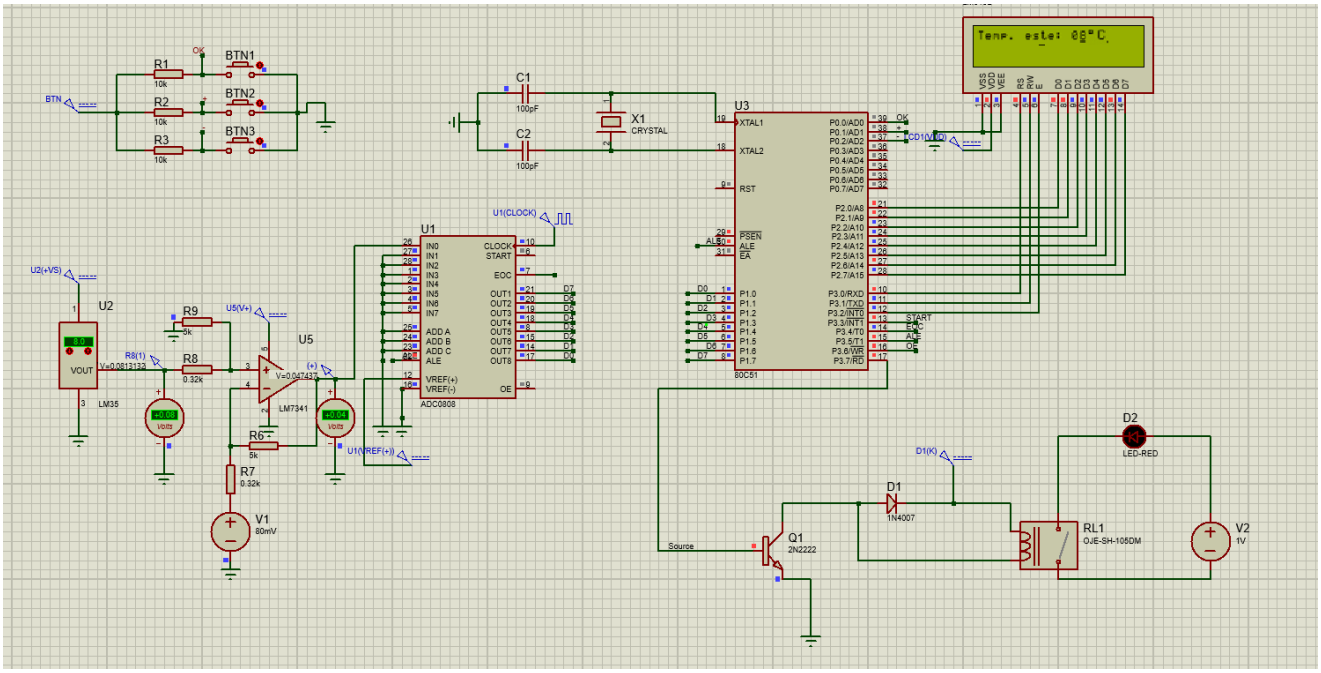


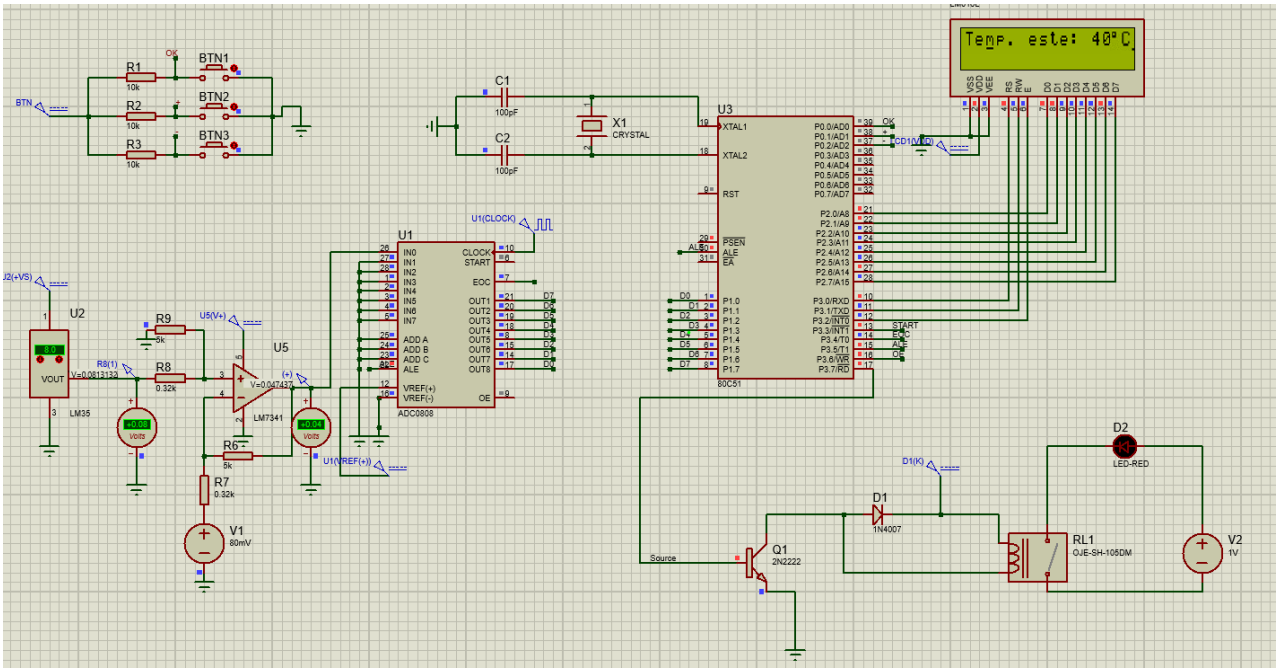
The final schematic with the code in C:





The schematic with the code in ASM:





4. 4. Bibliography:

- https://www.geeksforgeeks.org/comparison-of-8051-with-its-other-family-members/
- https://randomnerdtutorials.com/dht11-vs-dht22-vs-lm35-vs-ds18b20-vs-bme280-vs-bmp180/
- https://www.techtarget.com/whatis/definition/LCD-liquid-crystal-display
- https://en.wikipedia.org/wiki/MCS-51
- chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.ti.com/lit/ds/symlink/lm35.pdf

5. Appendix

Codul in C:

```
#include <reg51.h>

// Set the memory location of the program

// Initialize the pins
sbit RS = P3^0; // P3.0 pin is connected to RS pin of LCD
sbit RW = P3^1; // P3.1 pin is connected to RW pin of LCD
sbit EN = P3^2; // P3.2 pin is connected to EN pin of LCD
sbit START = P3^3; // P3.3 pin este is connected to START pin of the ADC
sbit EOC = P3^4;// pinul P3.4 este conectat la pinul EOC de la ADC
sbit ALE = P3^5; // pinul P3.5 este conectat la pinul ALE de la ADC
sbit OE = P3^6; // pinul P3.5 este conectat la pinul OE de la ADC

sbit P0_0 = P0^0;
sbit P0_1 = P0^1;
sbit P0_2 = P0^2;

sbit RELAY = P3^7; // P2.3 pin is connected to the relay control

// Use R3 to set an initial temperature reference value
unsigned char R3 = 21;
unsigned char R0,R1;
unsigned int temp[2], tem;
float t;

// Function prototypes
void delay();
void LCD(unsigned char a);
void write(unsigned char cmd);
void date_lcd(unsigned char a);
void adc();
void debouncing();
void convert();

void main() {
```

```
// Set the initial state of P0 pins
P0_0 = 1;
P0_1 = 1;
P0_2 = 1;

// Initialize the LCD
LCD(0x38); // afisare caractere pe o matrice de 5x7
delay();
LCD(0x0E); // display on, cursor on
delay();
LCD(0x01); // stergere dispaly(sterge si ce se afla in DDRAM)
delay();
LCD(0x06); // ENTRY MODE shift-eaza cursorul catre stanga
delay();

// Display "Temp. este:" on the LCD
LCD(0x80); // cursorul incepe de la prima linie
delay();
write('T');
delay();
write('e');
delay();
write('m');
delay();
write('p');
delay();
write('.');
delay();
write(' ');
delay();
write('e');
delay();
write('s');
delay();
write('t');
delay();
write('e');
```

```
delay();
write(':');
delay();
write(' ');
delay();

// Set P1 as input port
P1 = 0xFF; // Setam portul P1 ca si port de intrare
P0 = 0x00;  // Clear P0.0, P0.1, and P0.2
RELAY = 0;  // Ensure relay is off at the start

while (1) {
   // Read the ADC and convert the value to display on the LCD
   adc();
   convert();
   LCD(0x8E);  // Send command to LCD to display temperature label
   delay();
   date_lcd(temp[0]); // Send temperature value to LCD
   delay();
   LCD(0x8F);  // Send command to LCD to display "C" symbol
   delay();
   date_lcd(temp[1]); // Send "C" symbol to LCD
   delay();

   // Check if buttons are pressed
   if (P0_0 == 0) {
      // Reset button is pressed
      debouncing();
      R3 = 21; // Reset the temperature value to 21
   }
   if (P0_1 == 0) {
      // Plus button is pressed
      debouncing();
      R3++; // Increment temperature value
   }
   if (P0_2 == 0) {
      // Minus button is pressed
```

```
            debouncing();
            R3--; // Decrement temperature value
        }

        // Turn on the relay if temperature exceeds threshold (e.g., 40 degrees)
        if (temp[0] >= '4' && temp[1] >= '0') { // Check if temperature >= 40
            RELAY = 1; // Turn on the relay
        } else {
            RELAY = 0; // Turn off the relay
        }
    }
}

void delay() {
    unsigned int i;
    for (i = 0; i < 200; i++);
}

void LCD(unsigned char a) {
    P2 = a;     // Copy register A into P2
    RS = 0;     // RS=0 for command
    RW = 0;     // R/W=0 for write
    EN = 1;     // E=1 to activate the EN pin
    delay();    // Wait for a short delay
    EN = 0;     // Set EN pin back to 0 to deactivate it    // Return from subroutine
}

void write(unsigned char cmd) {
    P2 = cmd;  // Copy register A into P2
    RS = 1;  // RS=0 for command
    RW = 0;  // R/W=0 for write
    EN = 1;  // E=1 to activate the EN pin
    delay(); // Wait for a short delay
    EN = 0;  // Set EN pin back to 0 to deactivate it
}

void date_lcd(unsigned char a) {
```

```
    P2 = a;  // Copy register A into P2
    RS = 1;  // RS=1 for data
    RW = 0;  // R/W=0 for write
    EN = 1;  // E=1 to activate the EN pin
    delay(); // Wait for a short delay
    EN = 0;  // Set EN pin back to 0 to deactivate it
}

void adc() {
    START = 1;  // Start ADC conversion
    delay();// Wait for a short delay
    START = 0;  // Set START pin back to 0 to stop conversion
    while (EOC == 0);// Wait for EOC pin to go high
    OE = 1;     // Set OE pin to 1 to enable
    delay();
    OE = 0;
}

void debouncing() {
    unsigned char R5;
    R5 = 5;
    while (R5 != 0) {
        if (P0_1 == 0 || P0_2 == 0 || P0_0 == 0) {
            R5--;
        }
    }
}

void convert() {
    t = (P1 * 0.0196078431372549 / 12.5);
    tem = t * 100;
    temp[0] = tem / 10 % 10; // Convert the tens digit to ASCII
    temp[0] += 0x30;
    temp[1] = tem % 10; // Convert the ones digit to ASCII
    temp[1] += 0x30;
}
```

Code in ASM::

```
; Set the memory location of the program
org 0000h

; Initialize the pins
RS equ P3.0 ;P3.0 pin is connected to RS pin of LCD
RW equ P3.1 ;P3.1 pin is connected to RW pin of LCD
EN equ P3.2 ;P3.2 pin is connected to EN pin of LCD
START equ P3.3 ;P3.3 pin este is connected to START pin of the ADC
EOC equ P3.4 ;pinul P3.4 este conectat la pinul EOC de la ADC
ALE equ P3.5 ;pinul P3.5 este conectat la pinul ALE de la ADC
OE equ P3.6 ;pinul P3.5 este conectat la pinul OE de la ADC
setb P0.0
setb P0.1
setb P0.2

RELAY equ P2.3 ; P2.3 pin is connected to the relay control

; Use R3 to set an initial temperature reference value
mov R3, #21

; Initialize the LCD
mov a,#38h ;afisare caractere pe o matrice de 5x7
acall LCD
acall delay
mov a,#0Eh ;display on, cursor on
acall LCD
acall delay
mov a,#01h ;stergere dispaly(sterge si ce se afla in DDRAM)
acall LCD
acall delay
mov a,#06h ;ENTRY MODE shift-eaza cursorul catre stanga
acall LCD
acall delay

; Display "Temp. este:" on the LCD
```

```
mov a,#80h ;cursorul incepe de la prima linie
acall LCD
acall delay
mov a,#'T'
acall write
acall delay
mov a,#'e'
acall write
acall delay
mov a,#'m'
acall write
acall delay
mov a,#'p'
acall write
acall delay
mov a,#'.'
acall write
acall delay
mov a,#' '
acall write
acall delay
mov a,#'s'
acall write
acall delay
mov a,#'e'
acall write
acall delay
mov a,#'t'
acall write
acall delay
mov a,#'a'
acall write
acall delay
mov a,#':'
acall write
acall delay
mov a,#' '
```

```
acall write
acall delay

; Set P1 as input port and start the main loop
mov P1,#0ffh ;Setam portul P1 ca si port de intrare

main:
clr P0.0
clr P0.1
clr P0.2
; Read the ADC and convert the value to display on the LCD
acall adc
mov R4, a
acall convert
mov a, #8Eh
acall LCD
acall delay
mov a, R0
acall date_lcd
acall delay
mov a, #8Fh
acall LCD
acall delay
mov a, R1
acall date_lcd
acall delay

; Check if buttons are pressed
jb P0.0,Buton_reset
jb P0.1,Buton_plus
jb P0.2,Buton_minus
sjmp main

Buton_plus:
mov R5,#5
debouncing_1:
jb P0.1,Buton_plus ;Verificam daca butonul daca butonul e apasat
```

```
djnz R5,debouncing_1;Decrementeaza r5
inc R3 ;Incrementeaza R4 adica creste temperatura
setb RELAY ; Turn on the relay
sjmp main ;Reluam functia main

Buton_minus:
mov R5,#5
debouncing_2:
jb P0.2,Buton_minus ;Verificam daca butonul daca butonul e apasat
djnz R5,debouncing_2;Decrementeaza r5
dec R3 ;Decrementeaza R3 adica scade temperatura
setb RELAY ; Turn on the relay
sjmp main ;Reluam functia main

Buton_reset:
mov R5,#5
debouncing_3:
jb P0.1,Buton_reset ;Verificam daca butonul daca butonul e apasat
djnz R5,debouncing_3;Decrementeaza r5
mov R3,#21d ;Reseteaza valoarea 21 pentru temperatura setata
clr RELAY ; Turn off the relay
sjmp main ;Reluam functia main

; LCD subroutine to send a command to the LCD display
LCD:
mov P2,a    ; Copy register A into P2
clr RS      ; RS=0 for command
clr RW      ; R/W=0 for write
setb EN     ; E=1 to activate the EN pin
acall delay ; Wait for a short delay
clr EN      ; Set EN pin back to 0 to deactivate it
ret         ; Return from subroutine

; LCD subroutine to send data to the LCD display
write:
mov P2,a    ; Copy register A into P2
setb RS     ; RS=1 for data
```

```
clr RW      ; R/W=0 for write
setb EN     ; E=1 to activate the EN pin
acall delay ; Wait for a short delay
clr EN      ; Set EN pin back to 0 to deactivate it
ret         ; Return from subroutine

; Subroutine to send data to the LCD display
date_lcd:
mov P2,a    ; Copy register A into P2
setb RS     ; RS=1 for data
clr RW      ; R/W=0 for write
setb EN     ; E=1 to activate the EN pin
acall delay ; Wait for a short delay
clr EN      ; Set EN pin back to 0 to deactivate it
ret         ; Return from subroutine

; Subroutine to introduce a small delay
delay:
mov R2, #255 ; Move 255 into R2
again: djnz R2, again ; Decrement R2 and jump to again label if not zero
ret         ; Return from subroutine

; Subroutine to convert a number to ASCII
convert:
mov b, #5   ; Move 5 into B
div AB      ; Divide AB and store result in B
mov B, #10  ; Move 10 into B
div AB      ; Divide AB and store result in B
add a, #30h ; Add 30h to A to convert to ASCII
mov R0, a   ; Move A into R0
mov A, B    ; Move B into A
add a, #30h ; Add 30h to A to convert to ASCII
mov R1, a   ; Move A into R1
ret         ; Return from subroutine

; Subroutine to read ADC value
adc:
```

```
setb START  ; Start ADC conversion
acall delay ; Wait for a short delay
clr START   ; Set START pin back to 0 to stop conversion
astp: jnb EOC, astp ; Wait for EOC pin to go high
setb OE     ; Set OE pin to 1 to enable output
acall delay ; Wait for a short delay
mov a, P1   ; Move P1 into A
clr OE      ; Set OE pin back to 0 to disable output
ret         ; Return from subroutine

end          ; End of code ------------------------------------------------------------
```